

기말 프로젝트 보고서

pi-flag-coach — 플래그풋볼 실시간 전술 코칭 & 팀 관리 AI 에이전트

- 저장소: <https://github.com/yonseiaie/pi-flag-coach>
- 한 줄 요약: Pi 하니스 위에 Skill · MCP · Pi Extension · Web UI를 결합해, 플래그풋볼(5인제) 코치가 작전을 직접 설계·검증하고 AI에게 분석받는 서비스형 프 로토타입.

1. 선택한 시나리오

제시된 20가지 중 「실시간 전술 감시 및 피드백」 과 「맞춤형 팀 관리 솔루션」 을 결합해 선택했다.

특정 데이터 소스를 감시하다 의미 있는 변화가 생기면 알려 주는 Pi Agent의 특성을 활용해 상대 수비 지표를 모니터링하고 최적 플레이를 제안하며(전술 감시·피드백), 동시에 선수 라인업·훈련 일정·플레이북을 하나의 웹에서 관리한다(팀 관리). 종목은 선수 5명(X·Y·Z·C·QB)으로 구성되는 플래그풋볼로 구체화했다.

2. 문제 정의

플래그풋볼 코치는 경기·연습 중 짧은 시간에 플레이를 결정해야 하지만, 근거가 되는 데이터(상대 세이프티 깊이, 플렛 수비 성공률, 블리츠 빈도, 자기 팀 선수 스탯, 작전별 루트)가 흩어져 있고 수작업 조화가 느리다. 또한 일반 챗봇에 질문하는 방식은 (1) 사람이 데이터를 직접 복사해 넣어야 하고, (2) 답변 형식이 매번 달라 재현이 안 되며, (3) 근거 없는 환각 위험이 있고, (4) 외부 시스템에 기록되지 않는다.

특히 코치가 머릿속으로만 그리던 작전을 **시각적으로 그리고, 즉시 검증하고, AI에게 평가받는** 도구가 없다는 점이 핵심 문제였다. 따라서 데이터를 스스로 읽고(도구), 절차를 지키며(스킬), 외부 DB와 연동하고(MCP), 실시간 지표를 주입(Extension)하는 에이전트형 코칭 도구가 필요하다.

3. 서비스 대상 사용자

- 헤드코치 / 오픈스 코디네이터 — 작전 설계, 플레이 콜 결정, 전술 수정.
- 선수 관리 담당 — 경기별 라인업 구성, 선수 약점 기반 훈련 설계.
- 1차 타깃은 전담 분석관이 없는 아마추어~세미프로 팀(예: 대학 동아리 팀)이다. 실제 팀 데이터(연세 이글스 라인업·연습 스케줄, 상대 동탄/베이글스 경기 기록)를 샘플로 사용했다.

4. 핵심 기능

작동하는 웹 작전 보드를 중심으로 한다(장식이 아니라 실제 동작).

- 작전 설계** — 선수 5명 드래그 배치, 라우트 직접 그리기(자유 곡선) + 프리셋 12종(go/slant/out/in/cross/curl/comeback/corner/post/skinny-post/flat/hitch).
- 공 흐름 표시** — 볼 캐리어 지정(🎯 아이콘 + QB→캐리어 핸드오프/피치 점선), 페이크 핸드오프(회색 "fake" 점선). 스qip·핸드오프 작전 표현에 필요.
- 얼라인(포메이션)** — Spread/Bunch Back/Stack 및 각 Flip 등 기본 포메이션 + **사용자가 직접 만든 얼라인 저장·재사용**.
- 커스텀 라우트 저장** — 내가 그린 라우트를 이름 붙여 저장 후 어떤 선수에게나 프리셋처럼 적용.
- 재생** — 그린 라우트대로 선수가 움직이는 애니메이션으로 작전 검증.
- 플레이북** — 실제 작전(#11 X QB Pitch, #28 X motion Corner)과 얼라인을 카드로 제공, 클릭 시 보드로 로드. 저장/불러오기 + JSON 내보내기/가져오기.
- 선수 관리** — 4경기 공·수 라인업(전반/후반)을 편집 가능한 표로 관리.
- 훈련 계획** — 실제 연습 스케줄을 편집·행 추가/삭제 가능한 표로 관리.
- Pi 분석/평가** — 그린 작전을 JSON으로 직렬화해 에이전트에 보내 강점·약점·보완 평가.
- 실시간 수비 분석·훈련 자동 생성·멀티 에이전트** — 스킬·서브에이전트 기반 코칭 보조.

5. 시스템 구조

```
[웹 대시보드 web/index.html]
├── POST /api/chat ── 사용자 프롬프트가 에이전트를 깨움(triggerTurn 패턴)
└── ▼
[브리지 서버 server/bridge.mjs (Express)]
├── spawn pi -p (프롬프트는 stdin 으로 전달), 응답 relay
└── ▼
[Pi 에이전트 코어] ── AGENTS.md 프로젝트 지침
├── Skills   : player-stats · analyze-defense · training-planner · play-review
├── Extensions : flag-tools(커스텀 도구) · tactical-monitor(실시간 주입+위젯+알림)
├── MCP      : stats-db(sqlite→postgres) · team-store(filesystem)
├── Subagents : defense-analyst // offense-strategist
└── ▼
응답 → 브리지 → 웹 UI(작전 평가/설명 갱신)
```

데이터 계층은 `data/` (`roster.json`, `opponents.json`, `plays.json`, `lineups.json`, `practice-schedule.json`)이며 MCP를 통해 통계 DB로 승격할 수 있다. 에이전트 루프·세션 저장(`~/pi/agent/sessions/`)·LLM 호출은 전부 pi 코어가 담당하고, 우리는 코어를 수정하지 않고 능력(스킬·확장·MCP)만 얻는 pi 철학을 따랐다.

6. Skill / MCP / Pi Extension 활용 방식

6.1 Skill (`skills/<name>/SKILL.md`)

프론트매터의 `name/description`으로 LLM이 필요할 때만 로드한다(`progressive disclosure` → 컨텍스트 오염 최소화).

- **player-stats** : 선수/포지션 질문 시 로드. 세부 매치업은 `references/positions.md` 로 분리해 추가 토큰을 아낀다.
- **analyze-defense** : 세이프티 값이 $\geq 70\%$ → 덩 패스, 플랫 성공률 $\leq 50\%$ → 플랫/슬롯, 블리스 $\geq 65\%$ → 핫 루트 등 **수치 임계값 규칙**으로 추천을 결정(추측 금지·근거 강제).
- **training-planner** : 약점 지표 → 드릴 매핑으로 60~90분 세션을 자동 설계하고 도구로 저장.
- **play-review** : 웹 보드가 보낸 작전 JSON(선수 위치·라우트·프릿셋·볼 캐리어·페이크)을 받아 강점/약점/보완을 평가. "이 작전 분석시키기" 버튼이 호출.

6.2 Pi Extension (`extensions/*.ts`)

`export default function(pi: ExtensionAPI)` 팩토리에서 도구·이벤트를 등록한다.

- **flag-tools.ts** : `pi.registerTool` + `typebox` 스키마로 `list_players` / `get_player_stats` / `read_opponent_record` / `save_play` / `create_training_session` 5개 팀 관리 도구를 등록. 도구가 직접 `data/*.json` 을 읽고 써서 "팀 관리"를 수행한다.
- **tactical-monitor.ts** : `session_start` 에서 감시 위젯을 켜고, **before_agent_start** 이벤트로 매 턴 **직전 현재 수비 지표**를 메시지로 주입(실시간 감시), `ctx.ui.setWidget` / `setStatus` 로 전략 힌트 상시 표시, `coaching_report` 도구와 `/notify` 백그라운드 알림을 제공.

6.3 MCP (`.mcp.json`)

무인종으로 즉시 실행되는 공식 서버를 기본값으로 두어 채점·시연이 바로 가능하게 했다.

- **stats-db** : `@modelcontextprotocol/server-sqlite` — 경기 통계 DB. 운영 시 `_production_swap` 의 `server-postgres` 로 정의만 교체하면 구조 변경 없이 실제 DB에 연결된다.
- **team-store** : `@modelcontextprotocol/server-filessystem` — 팀 데이터(`data/`) 읽기/쓰기.

에이전트는 MCP로 통계 DB를 직접 조회해 상대 기록·라인업의 근거를 확보한다(외부 파일/DB 연동 요건 충족).

6.4 멀티 에이전트 오케스트레이션 (`.pi/agents/`)

중앙 에이전트가 `defense-analyst` (수비 약점 분석)와 `offense-strategist` (공격 콜 설계) 서브에이전트에 병렬 위임 → 컨텍스트 격리 + 속도 향상. 역할별 에이전트 운용 사례.

7. Web UI 설명

`web/index.html` 단일 페이지(빌드 불필요), 어두운 테마, 좌측 내비게이션으로 4개 뷰 전환.

- **작전 보드** : SVG 인터랙티브 보드. *그리기* 모드에서 선수를 끌면 라우트가 그려지고, *프릿셋*으로 정형 라우트를 자동 생성하며, *이동* 모드로 선수를 배치한다. *얼라인* 드롭다운(기본+내 얼라인), *공 지정/페이크 핸드오프* 버튼, *재생*(`requestAnimationFrame`), *저장/불러오기*를 제공. 우측에 수행 역할·자동 작전 설명·Pi 분석 결과 패널.
- **플레이북** : 실제 작전과 얼라인을 미니 필드 카드로 표시, 클릭 시 보드로 로드.
- **선수 관리** : 경기별 공·수 라인업 편집 표(저장·JSON 내보내기·기본값 복원).
- **훈련 계획** : 연습 스케줄 편집 표(행 추가/삭제·저장·내보내기).

UI가 "이 작전 분석시키기"를 누르면 현재 작전이 JSON으로 직렬화되어 `/api/chat` → 브리지 → Pi 에이전트(`play-review` 스킵)로 전달되고, 평가가 우측에 표시된다. 사용자 입력(작전 그리기/채팅)과 결과 확인(평가·설명)이 한 화면에서 이뤄지는 대화형 협업 루프다. API 키가 없으면 휴리스틱 데모로 폴백해 시연이 끊기지 않는다.

8. 구현 결과

- Pi 레퍼런스(`pi-work-agent`)와 동일한 디렉터리 규약(`extensions`·`skills`·`.mcp.json`·`.pi/agents`·MIT)을 따랐다.
- **스킬 4개 / 커스텀 도구 6개 / MCP 서버 2개 / 서브에이전트 2개 / 멀티뷰 웹앱 1개**를 구현했다.
- 브리지 서버를 통해 웹의 "Pi 분석" 버튼이 **실제 Pi 에이전트(claude)로 동작함**을 확인했다(데모가 아닌 실연결). 윈도우 환경에서 pi 실행·프롬프트 stdin 전달·응답 단일 전송 처리까지 안정화했다.
- 웹 UI는 실제 작전집·라인업·연습표를 반영했고, API 키 유무와 무관하게 렌더링되며 모든 편집 내용이 브라우저에 저장된다.

9. 한계점 및 개선 방향

한계	개선 방향
라우트 거리는 픽셀→야드 근사값	실제 필드 스케일 보정, 야드 라인 기준 보정
브리지가 print 모드(요청당 호출)	pi RPC/SDK 모드 전환 → 스트리밍·연속 세션

실시간 지표가 샘플 데이터	라이브 스코어/트래킹 API를 MCP로 연결해 진짜 실시간화
analyze-defense 임계값이 고정 규칙	과거 경기 학습 기반 동적 임계값으로 개인화
작전 저장에 브라우저 localStorage 중심	MCP/DB 저장으로 다중 사용자·기기 공유
MCP 쓰기 권한	역할별 권한 제한, 신뢰 설정 적용

10. (부록) 단순 프롬프트 대비 에이전트의 가치

항목	단순 프롬프트	본 에이전트
데이터 접근	사람이 복사·붙여넣기	도구로 스스로 조회
일관성	매번 형식 다름	스킬이 형식·규칙 고정
근거	환각 위험	수치 임계값·MCP DB 근거
외부 연동	수동	MCP로 직접 읽기/쓰기
추적	대화 닫으면 소멸	세션 JSONL 저장·재개
협업	텍스트만	웹 UI에서 그리고 즉시 평가받는 루프